

Baseline 单 Path0 为什么把 4 张 NPU 拖到 76.23%

从输入构造、有限搜索到闭环相位锁定的新手教程

基于 qos_storage_sim 的可复现实验

2026-09-05

目录

1 先给答案：你的直觉只错在最后一步	1
2 先审计问题中的前提	3
2.1 data 不是概率分布	3
2.2 “40 GB/s” 存在十进制与二进制单位歧义	3
2.3 “所有 NPU 都在工作” 必须定义	3
2.4 中间 1 秒不能让 Layer 0 完全“没关系”	3
2.5 “最低” 只在明确的有限搜索域内成立	3
3 新手需要的五个基本概念	3
3.1 一层是怎样运行的	3
3.2 Baseline 的 Path0 是什么	4
3.3 本文所说的“流带宽”是什么	4
3.4 能否用 Path 当前压力预判 deadline	4
3.5 NPU 利用率怎样算	5
4 我是怎样构造并找到这组输入的	5
4.1 第一步：先构造“受害者 + 大象 + 节拍器”	5
4.2 第二步：保证连续输入和稳态测量	6
4.3 第三步：固定三条大流，细搜短流 NQL	7
4.4 第四步：用物理 trace 验证，而不是只看平均数	7
5 关键机制：29.8566 ms 的锁相周期	7
5.1 为什么等待没有继续把相位打散	8
5.2 跨请求预取为什么没有救回来	8
6 数字归因：究竟是谁让 NPU 等	9
6.1 为什么名义带宽小于 40，SSD 反而只有 86.91%	9
7 单 Path0 是否真的是关键原因：四 Path 对照	10
8 你的“错位会缓解”在什么条件下可能成立	10
9 容易忽略的变量、成本和偏差	10
10 如何复现和阅读证据	11
11 最终判断	11

1 先给答案：你的直觉只错在最后一步

你的直觉是：某一层的 I/O burst (突发) 让一张 NPU 等待；等待会推迟它的下一层，于是不同 NPU 的读取会逐渐错开；既然错开了，就不该一直相撞。

这个推理的前半段是对的，后半段不是必然结论。

事实 (本次 trace)：等待确实改变了后续层的释放时刻；但改变后的相位没有继续漂移，而是进入了一个约 **29.8566 ms** 的重复周期。也就是说，流被错开了，却被错开到一组固定相位上，之后同样的碰撞反复出现。这叫作**相位锁定**。

还要纠正一个词：这里不是 SSD “一直处于 burst”。测量窗口内 SSD 利用率是 **86.9105%**，仍有 **13.0895%** 的空闲时间。真正持续的是“突发—排队—等待”这一**结构反复出现**，而不是 SSD 从头到尾没有空隙。

在精确的中间 1 秒窗口内，Baseline 单 Path0 的结果是：

指标	结果
NPU 0 利用率	21.6560%
NPU 1 利用率	83.2484%
NPU 2 利用率	100.0000%
NPU 3 利用率	100.0000%
4 NPU 算术平均	76.2261%
SSD 利用率	86.9105%
SSD 实际服务吞吐	34.7642 GiB/s
固定画像的名义带宽和	39.8862 GiB/s

每张 NPU 在这 1 秒里都有 active request。对每张卡，compute 时间 + I/O barrier 时间 = 1000 ms；低利用率不是因为没给 NPU 0 请求，而是因为它有请求却在等数据。

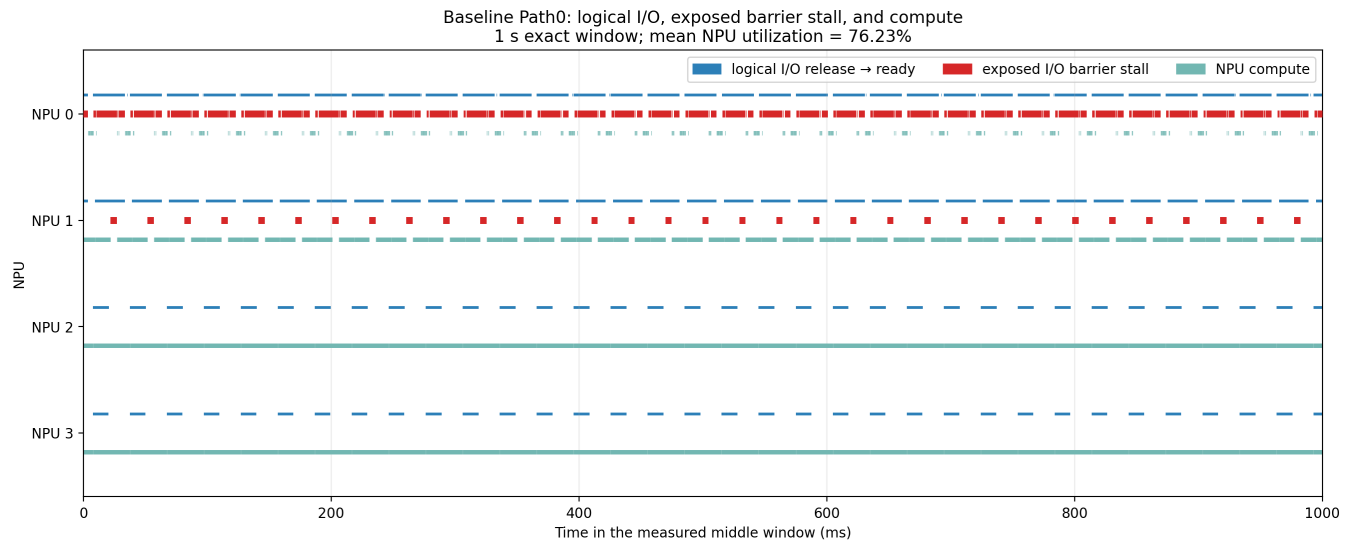


Figure 1: 完整 1 秒内，每张 NPU 的逻辑 I/O、barrier 和计算。NPU 0 的红色等待远多于绿色计算。

一句话结论：**等待会造成错位；但在“确定性计算时间 + 只提前一层预取 + 同一个 FCFS Path0”的闭环里，等待也会反馈到下一次释放时刻，因此错位可能收敛为固定周期，而不是自动变成均匀流量。**

2 先审计问题中的前提

在解释机制前，先把几个容易混在一起的概念拆开。

2.1 data 不是概率分布

仓库里的 data 是一个 84 点参数表：输入长度取若干离散值，NQL 取 64、128、256、512、1024、2048、4096。文件没有出现频率、概率权重或采样规则。因此，“满足 data 中的分布”不能严格解释成“按某个统计分布抽样”。

本实验采取的可审计解释是：

1. 保留 data 中 KV 大小、计算时间、TTFT 的关系；
2. 在相邻 NQL 点之间做分段线性插值；
3. 对 NPU 0 的 1K 短输入，沿 data 中 32K/48K 的长度斜率外推。

第 3 步是**模型构造**，不是实测数据。它适合构造反例，不足以直接声称真实 GLM-5.1 硬件也会得到 21.6560%。

2.2 “40 GB/s” 存在十进制与二进制单位歧义

模拟器历史字段名写作 _gb/_gbps，但 KV 大小按 2^{30} 字节归一化，实际数值单位是 GiB 和 GiB/s。

- 主结果：39.886158 GiB/s，满足模拟器数值上限 40 GiB/s；换成十进制是 42.827436 GB/s，**不满足字面量 40 GB/s**。
- 单位审计结果：另有一组输入为 37.172558 GiB/s = 39.913730 GB/s，平均 NPU 利用率仍只有 **77.7745%**。

所以，“能低于 80%”不依赖单位混用；但“主候选是否满足 40 GB/s”取决于你把上限解释成 GB/s 还是 GiB/s。

2.3 “所有 NPU 都在工作” 必须定义

若“在工作”是指 1 秒内始终在做计算，那么它和“平均利用率尽量低”互相矛盾。本实验采用的定义是：

- 每张 NPU 都有持续的 active request；
- 没有输入队列耗尽；
- 每张卡的整段时间都能归入计算或等待本层 I/O；
- 四张卡在窗口中分别完成了 47、8、4、4 个请求。

因此这里研究的是**有活却被阻塞**，不是人为不给某张卡任务。

2.4 中间 1 秒不能让 Layer 0 完全“没关系”

中间窗口确实排除了仿真最开始那个冷启动 Layer 0，这是对的。但每个请求只有 8 层，连续流每 8 层就进入一个新请求，仍会再次出现 Layer 0。跨请求预取还专门改变了新请求 Layer 0 的释放时刻。

更准确的说法是：**首次冷启动 Layer 0 不是这次低利用率的原因；请求边界上的 Layer 0 仍参与 Path0 排队，但核心机制也不依赖“它叫 Layer 0”，而依赖单 Path FIFO、突发释放和有限预取深度。**

2.5 “最低” 只在明确的有限搜索域内成立

76.2261% 是固定三条阻塞流后，对 NPU 0 的整数 NQL=153...192 共 40 个点统一复跑得到的最低点。它不是连续参数空间的数学全局最小值，也不是所有可能输入组合中的最低值。

3 新手需要的五个基本概念

3.1 一层是怎样运行的

对第 k 层，简化流程是：

1. 第 k 层数据已经到齐，NPU 才能开始算第 k 层；
2. **在第 k 层计算开始时**，提交第 $k+1$ 层的 I/O；
3. 第 k 层计算结束是第 $k+1$ 层数据的 deadline；
4. 若数据尚未到齐，NPU 进入 barrier，直到 I/O ready；
5. 若 k 是本请求最后一层，则在它开始计算时预取下一请求的 Layer 0。

因此正常层和跨请求 Layer 0 都只提前大约“一层计算时间”。这不是无限深预取，也不会预先缓存未来很多层。

3.2 Baseline 的 Path0 是什么

Baseline 对每一个物理块都返回 Path ID 0。Path0 内部是 FCFS：先进入队列的命令先服务。只有一个物理 SSD 后端，命令不可抢占。

这会产生经典的**队头阻塞**：NPU 0 只需要 7 个小块，但如果它们排在一千多个大流块后面，就不能因为“小而急”而越过队头。

为什么“等待导致错位”不等于“以后不会再撞车”

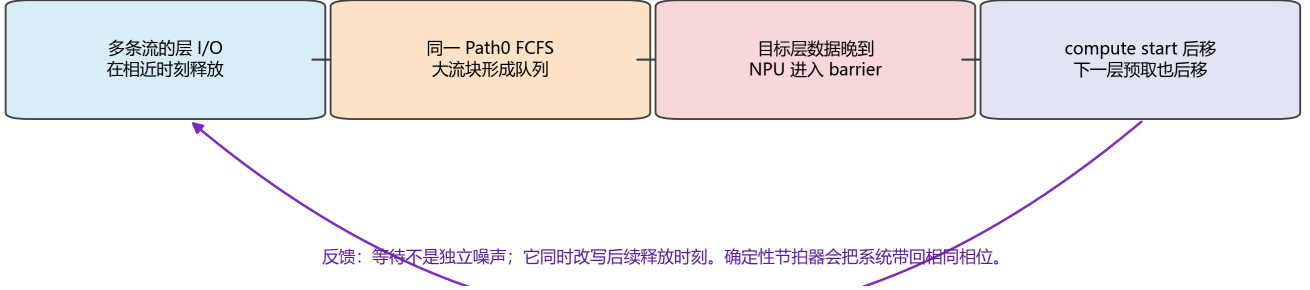


Figure 2: 等待如何通过下一层预取时刻反馈回同一 Path0。

3.3 本文所说的“流带宽”是什么

对每一条固定流，定义

$$B_i = \frac{V_i}{C_i/1000},$$

其中 V_i 是每层需要从 SSD 读取的 KV 大小 (GiB)， C_i 是每层理想计算时间 (ms)。等价地，若 C_i 用微秒表示，分母除以 10^6 。

它回答的问题是：**假如该流不被 I/O 阻塞，要在一层计算时间内读完下一层数据，长期平均需要多少带宽？**

它不是：

- 物理 I/O 在每个微秒的瞬时速率；
- Path0 为该流预留的 CIR；
- 发生等待后的实际吞吐；
- 把整层数据均匀摊在计算区间内的流体模型。

实际仿真会在 compute start 附近把下一层的大量块集中提交，因此物理到达是 burst。报告字段 `instantaneous_fixed_stream_sum_gibps` 应理解为“四条持续 active 的固定画像需求之和”，不能误读成 trace 上每一时刻真的有 39.8862 GiB/s 的平滑流量。

每个请求运行 8 层时，总 I/O 和总理想计算时间都会乘 8，所以 B_i 不变。

3.4 能否用 Path 当前压力预判 deadline

可以，但要先区分“便宜的估算”和“完整仿真”。对一个已经冻结的单 Path FCFS 队列，若目标层的全部命令确定排在现有 backlog 后面，且 SSD 只有带宽约束，则

$$T_{\text{ssd}} = \frac{Q_{\text{bytes}} + R_{\text{bytes}}}{40 \times 2^{30}} \times 1000 \text{ ms},$$

其中 Q_{bytes} 是 active 命令剩余字节与目标之前排队字节的总和， R_{bytes} 是本层仍需读取的字节数。若 deadline 还剩 D ms，则

$$\text{slack} = D - T_{\text{ssd}}.$$

$\text{slack} \geq 0$ 表示满足，数值就是提前量； $\text{slack} < 0$ 表示不满足， $-\text{slack}$ 是欠缺时间。

只给两个 I/O 数量是不充分的：100 个 4 KiB 命令和 100 个 176 KiB 命令相差 43 倍。若所有命令确实等大，补充一个 `io_size_bytes` 后才可以把数量换成字节。若大小不同，最好直接读取 Path 的 `pending bytes + active remaining bytes`；在纯带宽模型中不必知道每个命令的独立大小。

本文新增了两个纯函数：

```
from path_deadline_estimator import estimate_uniform_fcfs_path_deadline

estimate = estimate_uniform_fcfs_path_deadline(
    path_io_ahead=100,
    required_io_count=10,
    io_size_bytes=4096,
    time_budget_ms=1.0,
    bandwidth_gib_s=40.0,
    iops_limit=100_000,
)

print(estimate.meets_deadline) # False
print(estimate.shortfall_ms)  # 0.1 ms
```

它同时提供以字节为输入的 `estimate_fcfs_path_deadline()`，并输出预计 SSD 完成时间、排队时间、提前量和欠缺量。

如果真实设备有 IOPS 上限 $I_{\max}$ ，简化吞吐下界应改为

$$T_{\text{throughput}} = \max \left(\frac{Q_{\text{bytes}} + R_{\text{bytes}}}{B}, \frac{Q_{\text{io}} + R_{\text{io}}}{I_{\max}} \right).$$

40 GiB/s 的 4 KiB 命令需要 **10,485,760 IOPS** 才能跑满带宽，所以真实 4 KiB 场景几乎一定要知道 IOPS、固定命令延迟和队列深度。当前模拟器没有 SSD IOPS ceiling 或固定设备命令延迟；其普通满块是 **128 token × 1,408 byte = 180,224 byte = 176 KiB**，尾块可以更小。满块在 40 GiB/s 下服务约 4.196 μs ，对应约 238,313 IOPS。客户端另有 0.1 μs 的命令提交间隔，但它不是 SSD IOPS 参数。

这个便宜函数仍不能单独精确预测本模拟器最终的 `io_ready`，因为还可能需要：

- 当前 active 命令的**剩余**字节，而不是完整大小；
- 多 Path 时的 CIR/WRR 仲裁和其它 Path backlog；
- 本层尚未全部提交时，未来竞争命令怎样交错入队；
- SSD 后的 50 GiB/s NPU 接收链路及其队列；
- 真实设备的缓存、固定延迟、并行度和尾延迟分布。

因此，这个函数适合做快速 admission/routing 判断；命中边界或需要预测 NPU barrier 时，仍应使用完整离散事件仿真。

3.5 NPU 利用率怎样算

对 NPU i ：

$$U_i = \frac{\text{窗口内 compute 覆盖时间}}{1000 \text{ ms}}.$$

四卡平均是 $\bar{U} = (U_0 + U_1 + U_2 + U_3)/4$ 。I/O barrier 不计为 NPU 计算利用率，但它证明该 NPU 仍有 active work。

4 我是怎样构造并找到这组输入的

4.1 第一步：先构造“受害者 + 大象 + 节拍器”

目标不是简单把名义带宽堆到 40，而是让同一个 Path0 产生长尾排队，同时让至少两张卡有足够长的计算时间来隐藏自己的 I/O。

为避免在 A4 页面上压缩十列数据，输入身份和性能参数分成两张表。

NPU	角色	总 token	NQL	SSD token	物理块/层
0	短流受害者	1,024	169	855	7
1	大而快	196,608	368	196,240	1,534
2	大流节拍器 A	196,608	896	195,712	1,529
3	大流节拍器 B	196,608	896	195,712	1,529

NPU	KV/层 (GiB)	计算/层 (ms)	名义 B_i (GiB/s)	以 40 GiB/s 服务一层 (ms)
0	0.001121163	0.582149	1.925903	0.028029
1	0.257329941	12.390165	20.768888	6.433249
2	0.256637573	29.856563	8.595684	6.415939
3	0.256637573	29.856563	8.595684	6.415939

四条名义需求相加：

$$1.925903 + 20.768888 + 8.595684 + 8.595684 = 39.886158 \text{ GiB/s.}$$

设计意图如下：

- NPU 0 自己的 I/O 很小，本来 0.028 ms 就能读完；一旦它等待十几毫秒，几乎全部损失都能归因于排队而非自身数据量。
- NPU 1 每层也是约 0.257 GiB，但计算只有 12.390 ms，容易碰 deadline。
- NPU 2/3 输入完全相同、计算时间长达 29.8566 ms；它们自己的 I/O 能藏在计算后面，却会周期性地向 Path0 投入两组大块，成为稳定节拍器。

这里有一个容易忽略的变量：**每层数据量**和**长期平均带宽**不同。NPU 2/3 的名义带宽不算最高，但每次仍各自释放约 0.257 GiB，因此完全可以制造大 burst。

4.2 第二步：保证连续输入和稳态测量

四个 NPU 的请求在仿真开始时都已形成足够深的 backlog。由于短流完成得快，各 lane 预置请求数不相等：3,000、150、150、150。这是为了避免 NPU 0 先耗尽请求，并不改变每条 lane 的固定画像。

测量流程是：每张 NPU 先完成至少 8 个请求，再 settle 500 ms，随后只采集精确的中间 1,000 ms。窗口又切成 10 个 100 ms 小块，十块的四卡平均利用率都在 **75.0176% 到 77.3054%**，说明结果不是只靠 1 秒边界截到一个坏瞬间。

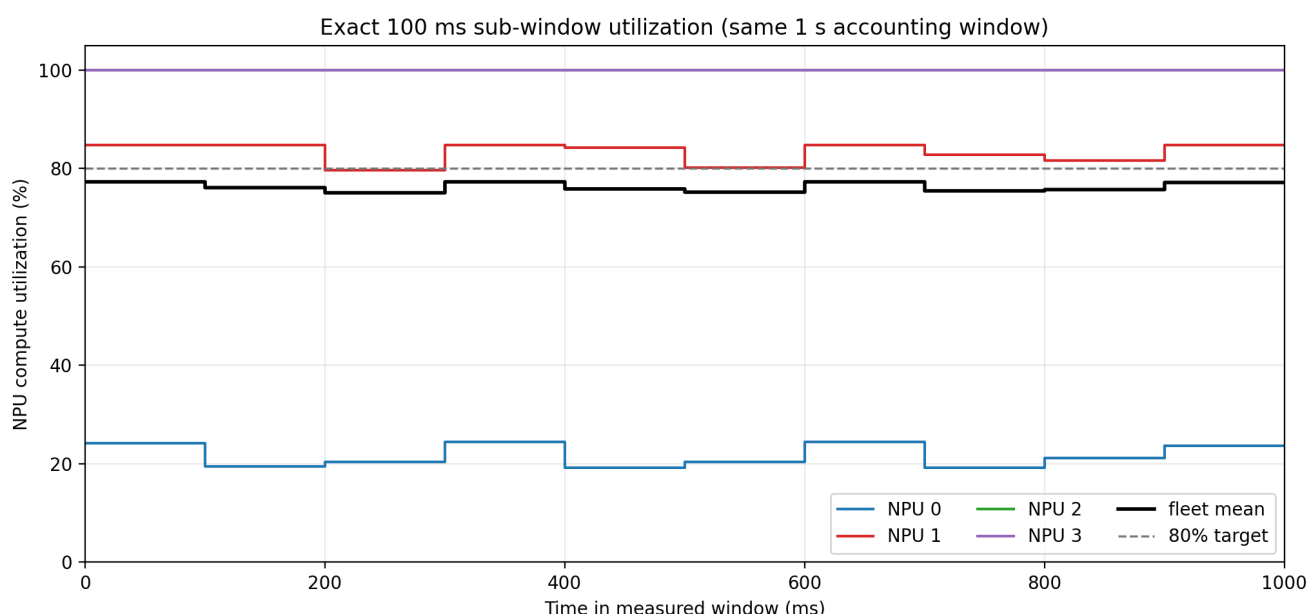


Figure 3: 十个 100 ms 子窗口的利用率。

4.3 第三步：固定三条大流，细搜短流 NQL

最后统一测量口径，对 NPU 0 的整数 NQL=153...192 逐点复跑。最低点出现在 NQL=169。

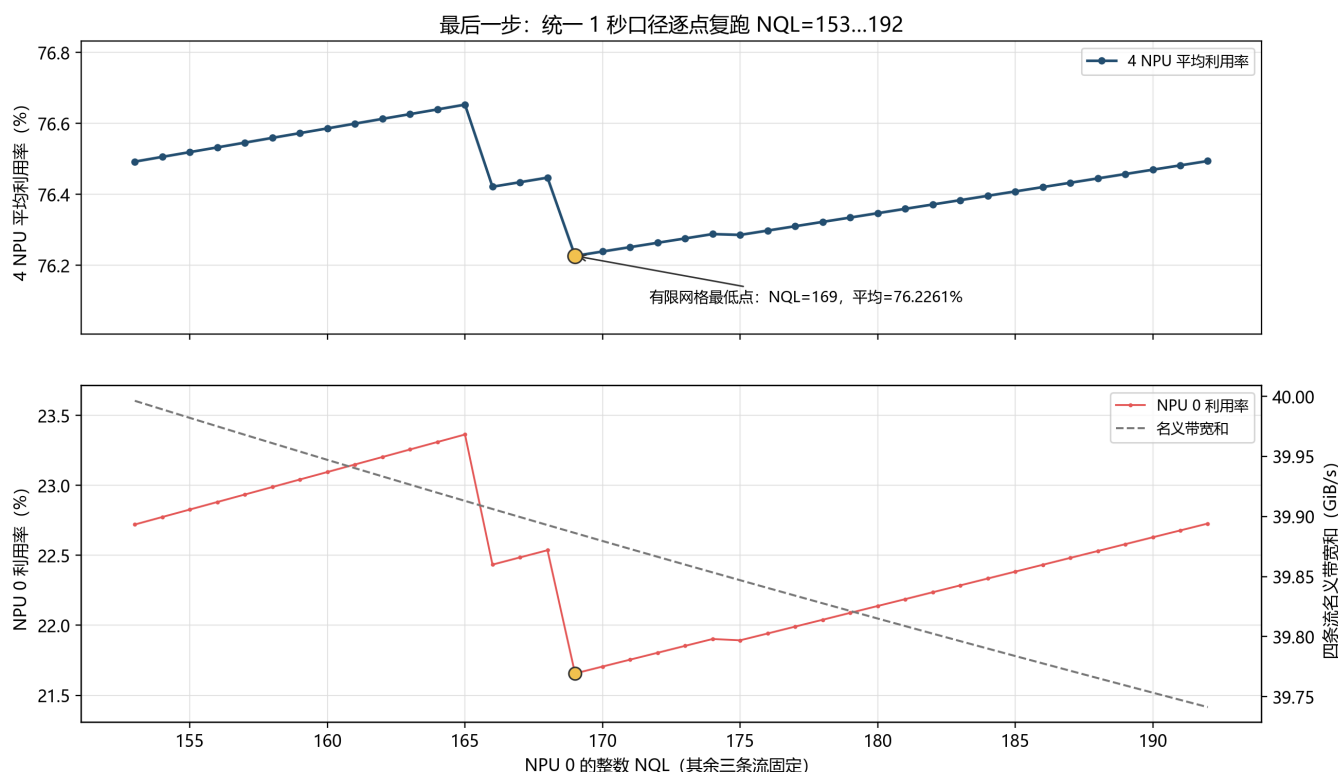


Figure 4: 有限网格搜索。曲线的跳变也提醒我们：相位问题不是“带宽越大就一定越差”的单调函数。

NQL=153 的名义带宽和更接近 40 GiB/s (39.9962)，平均利用率反而是 76.4921%，高于 NQL=169 的 76.2261%。因此“把带宽塞得越满，利用率一定越低”是错误前提。这里的主要变量是事件相位和队列顺序，不是带宽和本身。

曲线在 NQL=166 和 169 附近出现台阶。事实是事件 trace 的排序在这些点发生离散变化；把具体台阶完全归因于某一个固件规则还需要专门的边界事件对照，因此本文不作过度解释。

4.4 第四步：用物理 trace 验证，而不是只看平均数

最终运行同时记录了逻辑层和 207,297 个与窗口相交的物理块。验证结果包括：

- 所有观察到的物理命令都使用 Path0；
- Path0 的服务顺序与入队顺序完全一致，失配数为 0；
- SSD 同时最多服务一条不可抢占命令；
- 采集器开启前后的仿真摘要 SHA-256 完全相同，说明 observer 没有改变运行；
- 每张 NPU 的 compute + barrier 精确覆盖 1 秒；
- 没有 lane 的请求 backlog 耗尽。

5 关键机制：29.8566 ms 的锁相周期

下面把前 100 ms 放大。虚线是 NPU 3 的层 I/O 释放节拍；NPU 2 几乎同时释放。红色是 NPU 等数据，绿色是计算。

选第一个完整周期，时间均相对于 1 秒测量窗口起点：

1. **7.557 ms**：NPU 3 释放约 0.2566 GiB；**7.562 ms**：NPU 2 几乎同时释放同样大小的数据。
2. **7.589 ms**：NPU 0 释放一个只有 7 块的小层。它的一部分能插入早期队列，但仍到 **10.131 ms** 才 ready，超过 deadline，形成约 **1.959 ms** barrier。
3. NPU 0 在 10.131 ms 开始计算后才释放下一层。此时 NPU 2/3 的大量块以及 NPU 1 的大层已先入 Path0；这个小层直到 **26.881 ms** 才 ready，产生约 **16.169 ms** barrier。
4. NPU 1 在 **9.387 ms** 释放的大层也排在 NPU 2/3 后面，最终产生 **5.076233 ms** barrier。
5. **37.414 ms** 左右，NPU 3 再次释放下一层；同样结构重新开始。

Baseline 单 Path0 的计算、等待和物理服务

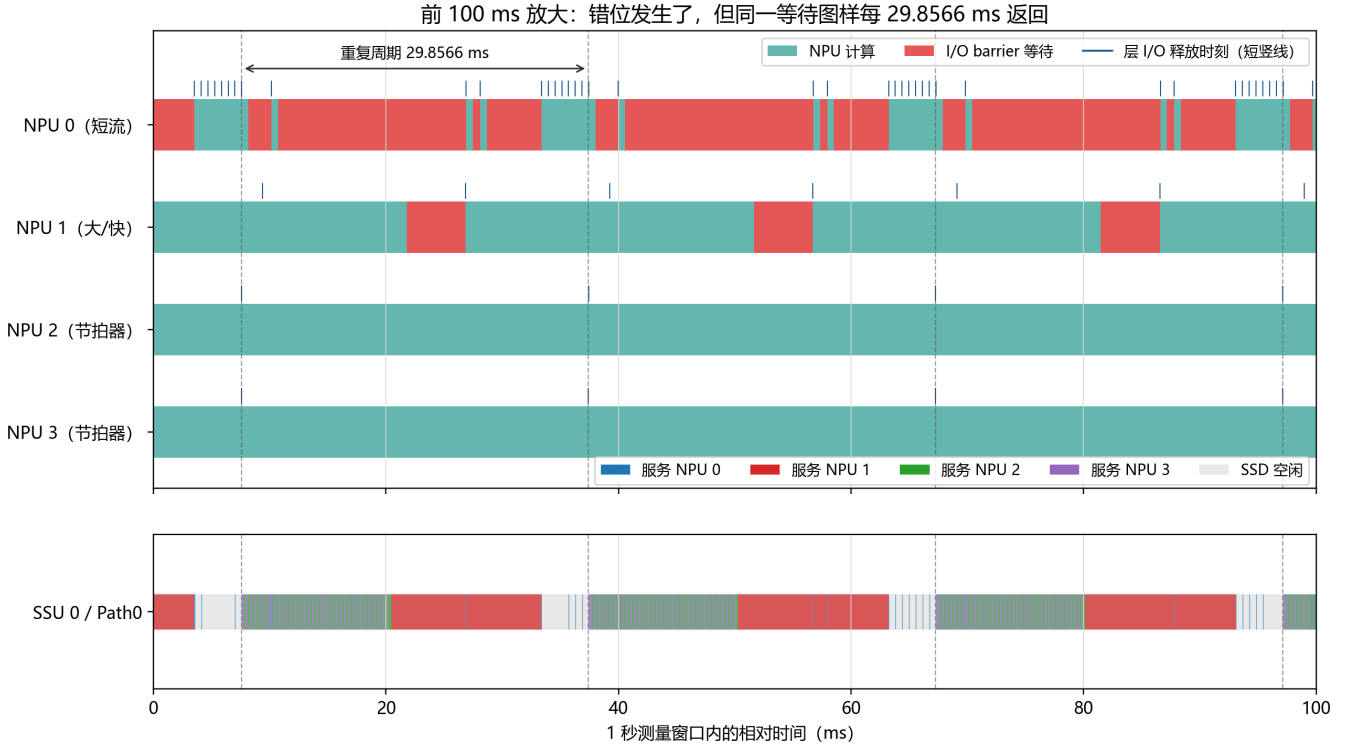


Figure 5: 前 100 ms 的放大图。底部是 Path0 的真实物理命令服务。

真正有说服力的不是“看起来差不多”，而是稳态 trace 中的时间恒等式。令

$$P = 29.856562816 \text{ ms.}$$

那么：

$$\begin{aligned} \text{NPU 2/3: } 1 \times 29.856562816 &= P, \\ \text{NPU 1: } 2 \times 12.390164802 + 5.076233212 &= P, \\ \text{NPU 0: } 11 \times 0.582149492 + 23.452918408 &= P. \end{aligned}$$

NPU 0 的 23.4529 ms 等待又由同一组约 1.959、16.169、0.629、4.696 ms 的 barrier 组成。下一周期里，这组等待以相同相位再次出现。

5.1 为什么等待没有继续把相位打散

原因不是“等待没有改变时刻”，而是它改变时刻的方式属于一个闭环：

1. 本层 I/O 晚到，使 compute start 后移；
2. 模拟器在 compute start 才提交下一层 I/O；
3. 所以同一段等待被复制进下一层释放时刻；
4. 下一层仍进入同一个有记忆的 FIFO 队列，不能越过已经排队的大块；
5. NPU 2/3 的确定性 29.8566 ms 节拍再次到来，把其它流拉回同一相对相位。

这与“每次等待都是独立随机扰动”的系统完全不同。独立随机扰动可能让相位扩散；这里的等待由同一个共享队列产生，又反过来控制未来到达，因此可能同步而不是去同步。

相位图把这个区别画得最直接：横轴是每个 29.8566 ms 周期内的位置，纵轴是周期编号。若相位持续漂移，点会形成斜线或云团；实际点形成稳定竖列。

5.2 跨请求预取为什么没有救回来

跨请求预取已经开启。它做的是：当前请求最后一层**开始计算时**，启动下一个已到达请求的 Layer 0 I/O。它有三个限制：

相位没有持续漂移：散点形成竖列，说明释放模式被锁在固定相位
(若等待能不断把流打散，竖列会逐周期向左右移动或扩散)

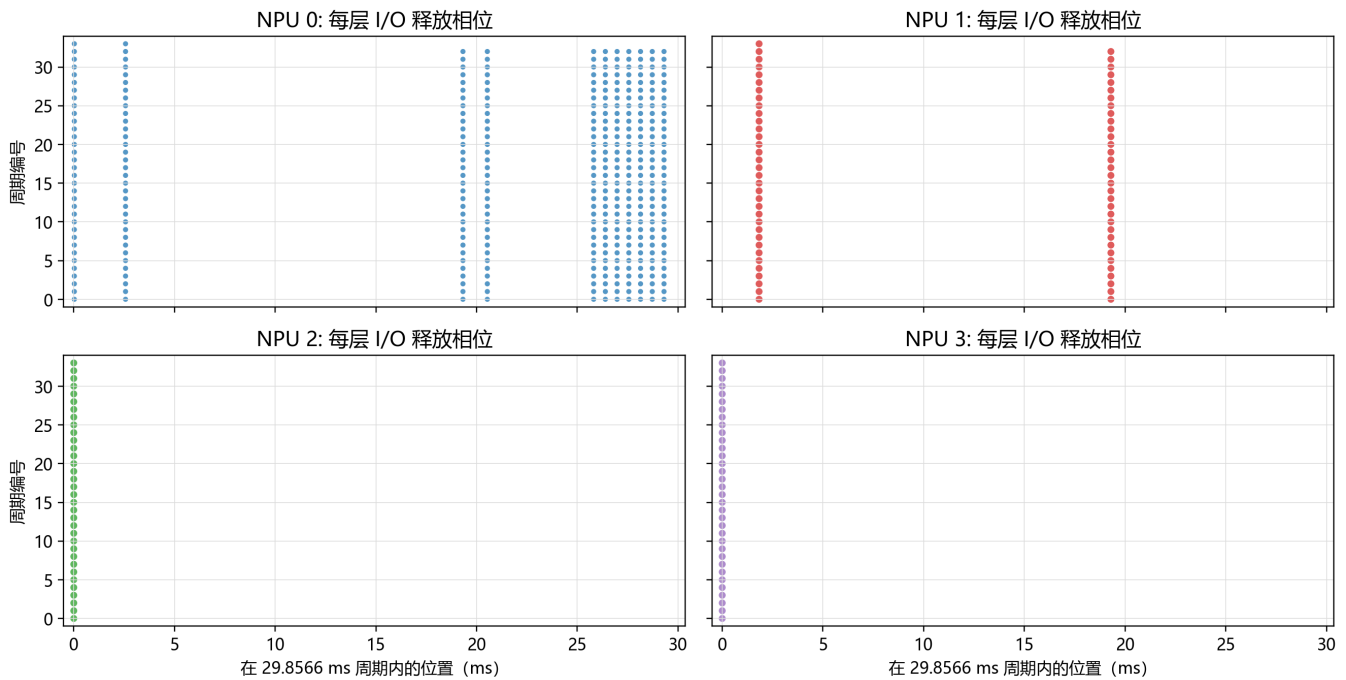


Figure 6: 1 秒内各流的层 I/O 释放相位。竖列说明等待后的相位固定，而非逐周期漂移。

1. 只预取下一个请求的 Layer 0，不会建立多层深缓冲；
2. 预取的块仍进入同一个 Path0，不能越过旧 backlog；
3. 触发时刻也跟随最后一层 compute start，因而仍在上述反馈环中。

NPU 0 的单层理想计算只有 0.582 ms，而它的典型坏尾排队可达约 16.7 ms。一层 lookahead 不可能覆盖这个量级的尾延迟。跨请求预取能消除一部分请求边界空洞，但它不是队列优先级，也不是多层缓存。

6 数字归因：究竟是谁让 NPU 等

NPU	compute (ms)	barrier (ms)	利用率	barrier 与其它 NPU 的 SSD 服务重叠 (ms)
0	216.560	783.440	21.6560%	779.590
1	832.484	167.516	83.2484%	0.107
2	1000.000	0.000	100.0000%	0.000
3	1000.000	0.000	100.0000%	0.000

NPU 0 的 barrier 有 **99.9879%** 与 SSD busy 重叠；按正在被服务的数据来源分解：

- 服务 NPU 0 自己：3.756 ms；
- 服务 NPU 1：389.656 ms；
- 服务 NPU 2：197.842 ms；
- 服务 NPU 3：192.092 ms。

这是“短流被其它流的块挡住”的直接证据。NPU 0 的物理块 queue wait 中位数只有 0.0205 ms，但 P95 是 16.7323 ms、P99/最大值约 16.7445 ms。**平均或中位数很小并不代表 deadline 安全；少量稳定出现的长尾足以让短计算层停很久。**

NPU 1 的情况不同：它的 barrier 期间主要是 SSD 正在服务它自己的数据（167.408 ms），说明它的单层预取窗口不足以完全覆盖自己的迟到层。不能把 NPU 1 的全部损失也归因于“其它 NPU 队头阻塞”。

6.1 为什么名义带宽小于 40，SSD 反而只有 86.91%

39.8862 GiB/s 是按**无等待的理想计算节拍**计算的名义需求。发生 barrier 后，计算开始变慢；而下一层 I/O 又必须等 compute start 才释放。因此生产者被反压，实际发出的层数减少。

在一个锁相周期内，近似出现：NPU 0 完成 11 层、NPU 1 完成 2 层、NPU 2/3 各完成 1 层。对应实际生成速率约为：

流	锁相后的近似 I/O 速率 (GiB/s)	名义需求 (GiB/s)
NPU 0	0.4131	1.9259
NPU 1	17.2377	20.7689
NPU 2	8.5957	8.5957
NPU 3	8.5957	8.5957
合计	34.8422	39.8862

这就是一个闭环吞吐下降：**先有 burst 排队和 deadline miss，随后 barrier 降低未来到达率，于是 SSD 可以出现空闲，但已经被锁住的 NPU 仍会周期性等待。**

所以 $\sum B_i < 40$ 只是理想流体模型里的必要速率条件，不是逐层 latency/deadline 保证。

7 单 Path0 是否真的是关键原因：四 Path 对照

为了做机制诊断，保持输入、物理 SSD、QoS 表、预取和测量方法不变，只把四个 NPU 固定映射到 Path 0、1、2、3。物理 SSD 仍然只有一个，并没有把硬件带宽乘四；变化只是调度器能从四个队头之间仲裁，不必让 NPU 0 的小块永远困在同一个长 FIFO 后面。

情况	平均 NPU 利用率	NPU 0	NPU 1	NPU 2	NPU 3
Baseline：全部 Path0	76.2261%	21.6560%	83.2484%	100%	100%
机制对照：四个独立 Path	95.8124%	100%	83.2495%	100%	100%

模型内的强因果推断：单 Path0 的 FCFS 队头阻塞是 NPU 0 低利用率的关键原因，因为只改变 Path 映射后，NPU 0 从 21.6560% 恢复到 100%。

不能夸大的地方：NPU 1 基本没有改善，说明共享物理 SSD 的 burst 形态和只有一层的预取深度也是共同原因。四 Path 对照支持单 Path0 的作用，但不能证明所有损失都只由 Path 数量造成。

8 你的“错位会缓解”在什么条件下可能成立

下面是预测，不是本次实验已经证明的事实：

- 各流有独立且足够大的到达抖动，等待不再由同一队列决定；
- 计算时长没有接近小整数倍关系，固定相位不容易闭合；
- 允许提前两层或更多层预取，用缓冲吸收 10-17 ms 尾延迟；
- 不同流进入可仲裁的不同 Path，短小紧急 I/O 能越过另一个 Path 的长队列；
- 物理调度采用 deadline-aware、公平轮转或可抢占机制；
- I/O 到达是独立 open-loop 流量，而不是由 compute start 触发的 closed-loop 流量。

即使满足其中一项，也只能说相位锁定风险可能下降，不能不经仿真就保证完全消失。

9 容易忽略的变量、成本和偏差

1. **1K 短流是外推画像。** data 最短为 32K；真实 1K GLM-5.1 的 kernel 启动、固定开销和 KV 行为可能不遵循线性外推。
2. **确定性偏差。** 所有请求到达时间为 0，使用固定 submit seed=42，没有服务时间噪声。这有利于观察锁相反例，但可能放大稳定周期。需要做 seed、初始 offset 和随机 jitter 扫描。
3. **8 层会提高请求边界频率。** GLM-5.1 官方配置是 78 层，而本实验按要求只执行 8 层。跨请求 Layer 0 每 8 层出现一次，不能直接外推到完整 78 层运行。
4. **上下文上限。** 官方配置的最大位置数为 202,752；本实验的 192K 即 196,608 token 在范围内。data 的 200K 若按 $200 \times 1024 = 204,800$ token 解释则超过该上限，所以没有选用。官方配置见 [zai-org/GLM-5.1 config.json](https://zai-org/GLM-5.1-config.json)。
5. **平均数掩盖公平性。** 76.23% 看似离 80% 不远，但其中两张卡满载、一张卡只有 21.66%；这是严重的 per-NPU 不公平。
6. **物理模型边界。** 结论是离散事件模拟器内部结果，没有包含真实 SSD 固件、缓存命中、PCIe 抖动、NPU kernel 调度等全部细节。

7. **有限搜索偏差**。只对 NPU 0 的 40 个整数 NQL 点做最终细搜，其余三条流固定。不要把“有限网格最低”写成“全局最坏”。
8. **带宽约束不是延迟约束**。若产品真正关心每层 deadline，应同时限制 burst size、queue wait P99 和 prefetch depth，不能只限制 $\sum B_i$ 。

10 如何复现和阅读证据

在仓库根目录执行：

```
python run_baseline_4npu_ssul_low_utilization.py \
  --output results/baseline_4npu_ssul_low_utilization
```

```
python make_baseline_path0_tutorial_assets.py
```

```
python -m unittest -v test_path_deadline_estimator.py
```

主要证据文件：

文件	用途
input_profiles.csv	四条固定流的长度、NQL、KV、计算时间和带宽
search_summary.csv	NQL=153...192 的完整有限搜索表
request_layer_timeline.csv	每层 I/O release/ready、deadline、compute 和 barrier
physical_block_trace.csv	与窗口相交的 207,297 个 Path0 物理块
simulator_summary.json	未裁剪的 Baseline 稳态统计
result.json	验证项、stall 归因、四 Path 对照和单位审计
path_deadline_estimator.py	单 Path frozen-FCFS 的快速 deadline/slack 估算函数
test_path_deadline_estimator.py	带宽、backlog 和 4 KiB IOPS 场景的单元测试

关键实现位置：

- policy_logic.py 的 baseline_path_ids()：所有 I/O 固定使用一个 Path；
- sim.py 的 PathQueue：FCFS 队列和不可越过的队头；
- continuous_batch_sim.py 的 _handle_compute_schedule()：在 compute start 预取下一层；
- 同文件的 _start_cross_request_layer0_prefetch()：最后一层开始时预取下一请求 Layer 0；
- run_baseline_4npu_ssul_low_utilization.py：输入构造、稳态测量、trace 和对照实验。

11 最终判断

事实：在这个确定性的 Baseline 模型中，等待确实让流错位，但 1 秒 trace 显示错位后形成固定的 29.8566 ms 相位列；NPU 0 有 783.440 ms barrier，其中 779.590 ms 与其它 NPU 的 SSD 服务重叠。

模型内因果结论：单 Path0 FCFS 是 NPU 0 被拖到 21.6560% 的关键条件；四 Path 对照把它恢复到 100%。

不是事实而是待验证预测：真实硬件中的随机抖动可能削弱这组精确锁相，也可能只把它变成概率性的长尾碰撞。只有对初始相位、seed、jitter、层数和预取深度做扫描后，才能判断该机制在真实 workload 中出现的频率。

因此，对你的问题“这里不行吗？”最直接的回答是：**这里会错位，但不够；因为错位由同一个共享 FIFO 闭环产生，它没有把相位随机化，反而把系统锁进了一个重复碰撞周期。**